Midterm Project- R Code and Outputs

Load in neccessary packages

```
library(tidymodels)
library(caret)
library(corrplot)
library(glmnet)
library(kableExtra)
library(earth)
```

Part I

Using this dataset, please help Researcher A build a prediction model for PFS, aiming to understand how demographic, tumor, and treatment characteristics influence progression-free survival in breast cancer patients.

To answer this question, your report should at least include the following:

Model Training: Provide a detailed description of the model training procedure and how you obtained the model. Your description should include sufficient detail so that another statistician can reproduce the same model. Points will be deducted if the details provided are insufficient to reproduce your results.

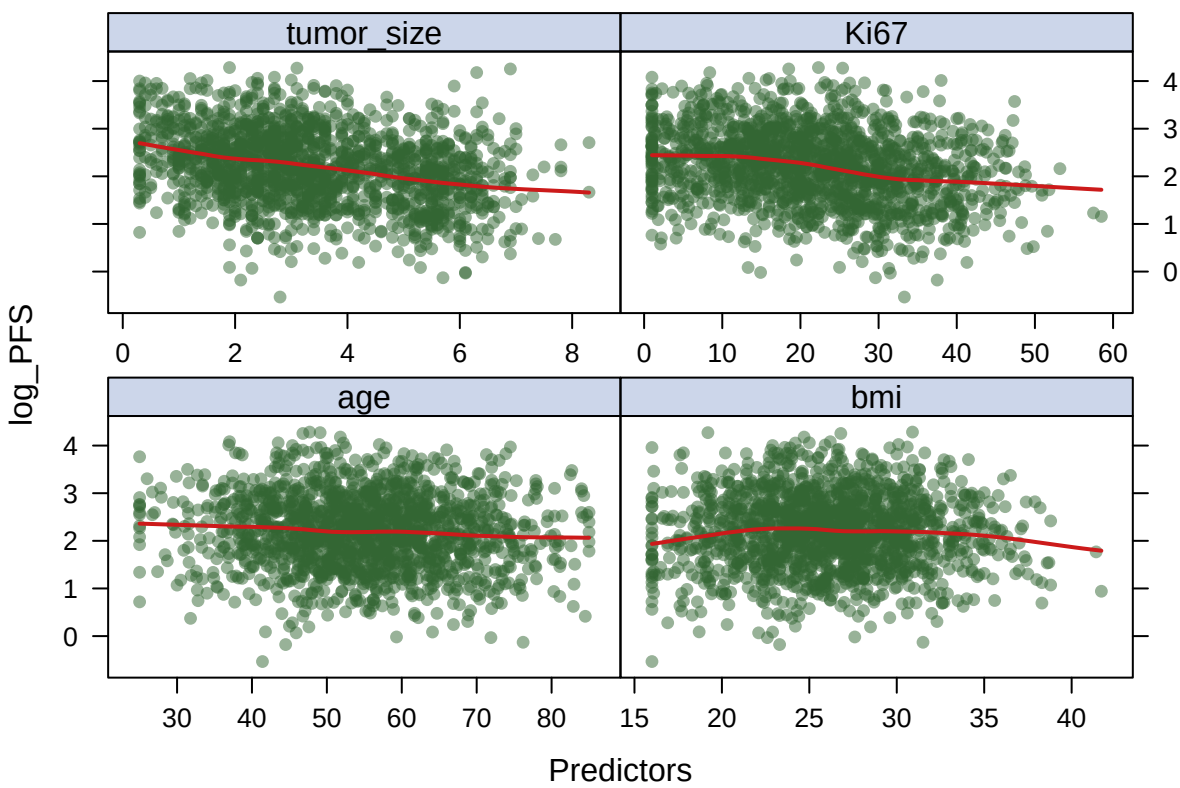
Import and partition data using 'tidymodels' package

```
load("data/datA.RData")

set.seed(2)

datA_split = initial_split(datA, prop = 0.8)

# Extract the training and test data
training_datA = training(datA_split)
testing_datA = testing(datA_split)

# Training data
x_trainA = model.matrix(log_PFS ~ . - id, training_datA)[, -1]
y_trainA = training_datA$log_PFS

# Test data
x_testA = model.matrix(log_PFS ~ . - id, testing_datA)[, -1]
y_testA = testing_datA$log_PFS

# Remove ID
```

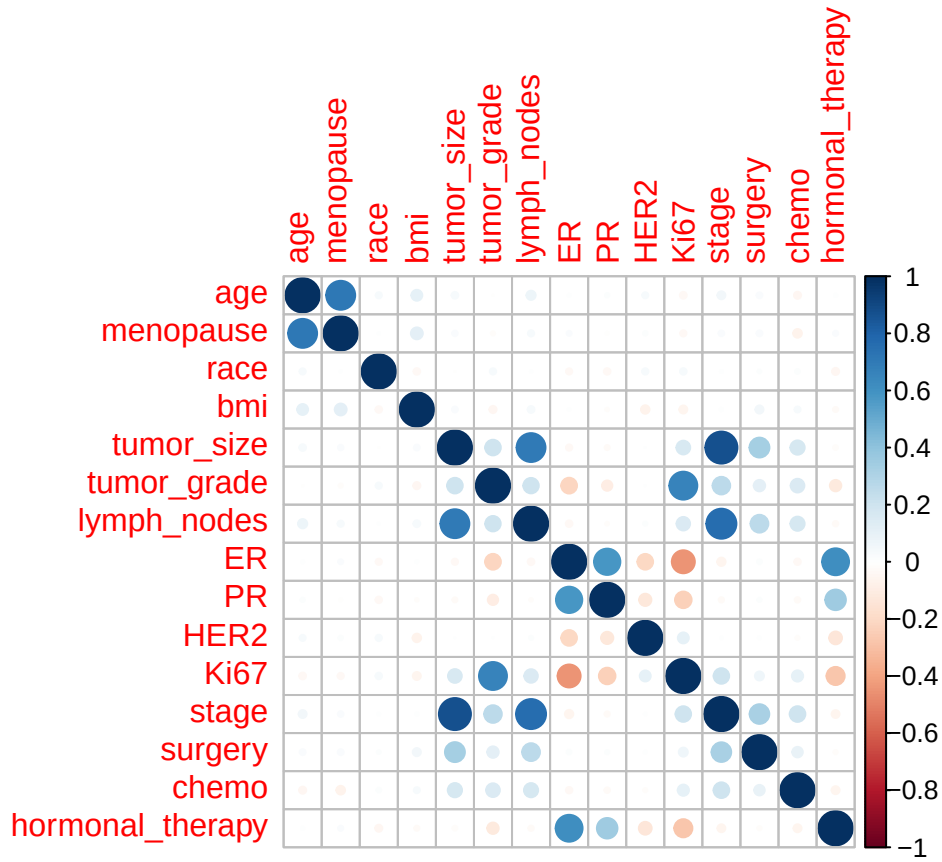
```r
training_datA <- training_datA |>  select(-"id")
testing_datA  <- testing_datA  |> select(-"id")
```

Exploratory Data Analysis

```r
# Changing the plot display
theme1 <- trellis.par.get()
theme1$plot.symbol$col <- rgb(.2, .4, .2, .5)
theme1$plot.symbol$pch <- 16
theme1$plot.line$col <- rgb(.8, .1, .1, 1)
theme1$plot.line$lwd <- 2
theme1$strip.background$col <- rgb(.0, .2, .6, .2)
trellis.par.set(theme1)

# Feature plots to look at trends between predictors and the outcome (log_PFS)
featurePlot(x = training_datA[ ,c(1, 4, 5, 11)],
            y = training_datA[ ,16],
            plot = "scatter",
            span = .5,
            labels = c("Predictors","log_PFS"),
            type = c("p", "smooth"),
            layout = c(2, 2))
```



```r
# Calculates the correlation matrix for all the predictors
corrplot(cor(x_trainA), method = "circle", type = "full")
```



This data set includes 7 binary predictors, 3 categorical predictors, 5 continuous predictors, and 1 continuous outcome in log scale.

Chose elastic net because continuous variables formed a cloud, meaning we could assume linearity? homoscedasticity?

Correlation plot revealed that certain variables were highly correlated. Chose to address this with elastic net because we can address colinearity for [] and [] reasons

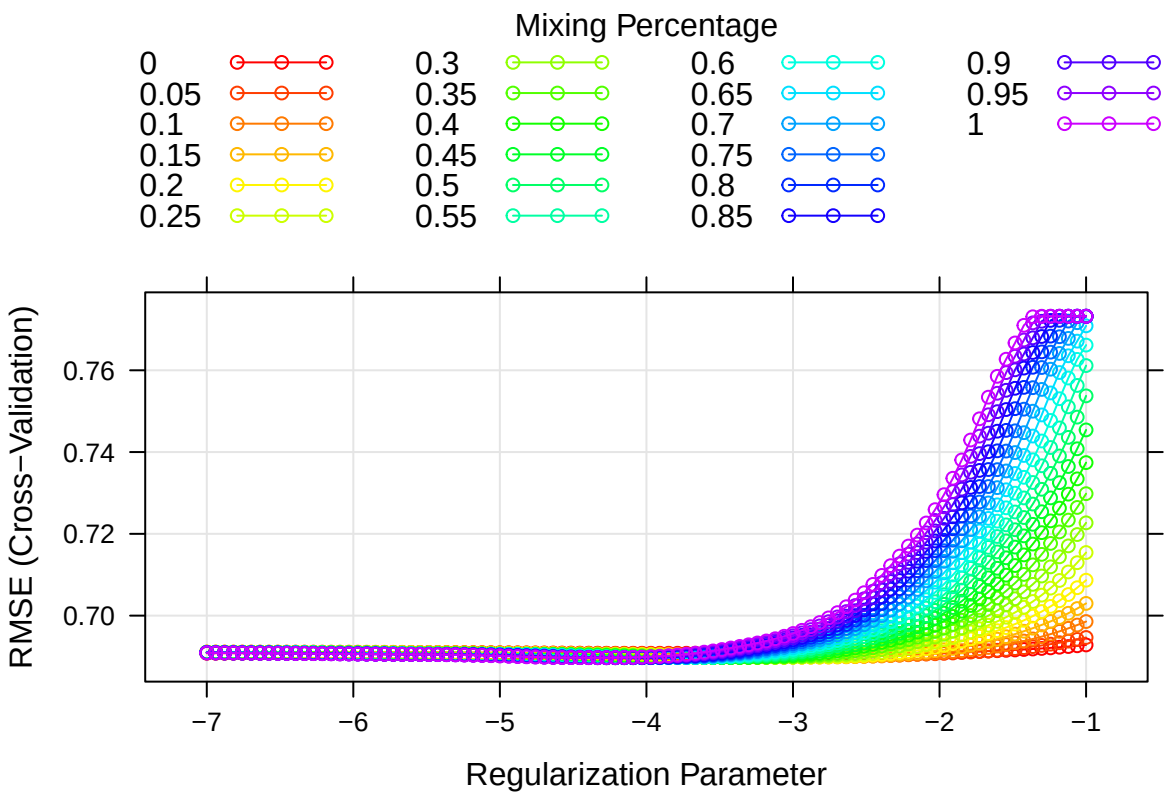
```r
# Elastic Net

set.seed(2)
ctrl1 <- trainControl(method = "cv", number = 10) # set up cv
set.seed(2) # set seed for reproduction

enet.fit <- train(log_PFS ~ .,
                  data = training_datA,
                  method = "glmnet",
                  tuneGrid = expand.grid(alpha = seq(0, 1, length = 21),
                                         lambda = exp(seq(-1, -7, length = 100))),
                  trControl = ctrl1)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```r
enet.fit$bestTune
```

```
##      alpha     lambda
## 2045     1 0.01312373
```

```r
myCol <- rainbow(25)
myPar <- list(superpose.symbol = list(col = myCol),
superpose.line = list(col = myCol))

plot(enet.fit, par.settings = myPar, xTrans = log)
```




```r
# Coefficients in the final model
coef(enet.fit$finalModel, enet.fit$bestTune$lambda)
```

```
## 16 x 1 sparse Matrix of class "dgCMatrix"
##                   s=0.01312373
## (Intercept)        2.775488586
## age               -0.002649168
## menopause          .
## race               .
## bmi                .
## tumor_size        -0.053637711
## tumor_grade       -0.080081328
## lymph_nodes       -0.054497860
```

```
## ER                 0.181544390
## PR                 .
## HER2              -0.177297122
## Ki67              -0.003930509
## stage              .
## surgery           -0.005056867
## chemo              .
## hormonal_therapy  0.160451277
```

Results: Report the model. Interpret the model and assess its performance.

```r
# Predict on testing_datA
enet.pred <- predict(enet.fit, newdata = testing_datA)

rmse_A <- RMSE(enet.pred, testing_datA$log_PFS)
r2_A  <- R2(enet.pred,   testing_datA$log_PFS)
```

```r
# Create table with the optimal tuning parameters for the chosen model
opt_para <- enet.fit$bestTune
rownames(opt_para) <- NULL
opt_para |>
  kable()
```

alpha	lambda
1	0.0131237

```r
# Create a table with the model coefficients
coefs1 <- as.data.frame(as.matrix(coef(enet.fit$finalModel, enet.fit$bestTune$lambda)))
coefs1$Predictors <- rownames(coefs1)
colnames(coefs1)[1] <- "Coefficients"
coefs1 <- coefs1[, c("Predictors", "Coefficients")]
rownames(coefs1) <- NULL
coefs1 |>
  kable()
```

Predictors	Coefficients
(Intercept)	2.7754886
age	-0.0026492
menopause	0.0000000
race	0.0000000
bmi	0.0000000
tumor_size	-0.0536377
tumor_grade	-0.0800813
lymph_nodes	-0.0544979
ER	0.1815444
PR	0.0000000
HER2	-0.1772971
Ki67	-0.0039305
stage	0.0000000
surgery	-0.0050569

Predictors	Coefficients
chemo	0.0000000
hormonal_therapy	0.1604513

```r
# Create a table with the interpretable output
coefs2 <- as.data.frame(as.matrix(coef(enet.fit$finalModel, enet.fit$bestTune$lambda)))
coefs2$Predictors <- rownames(coefs2)
colnames(coefs2)[1] <- "Percent Change in PFS"
coefs2 <- coefs2[, c("Predictors", "Percent Change in PFS")]
coefs2 <- coefs2 |>
  filter(Predictors %in% c("(Intercept)", "age", "tumor_size", "tumor_grade", "lymph_nodes",
                           "ER", "HER2", "Ki67", "hormonal_therapy"))
coefs2$`Percent Change in PFS`<- (exp(coefs2$`Percent Change in PFS`)-1)*100
rownames(coefs2) <- NULL
coefs2 |>
  kable()
```

Predictors	Percent Change in PFS
(Intercept)	1504.6465159
age	-0.2645662
tumor_size	-5.2224587
tumor_grade	-7.6958725
lymph_nodes	-5.3039465
ER	19.9067762
HER2	-16.2469101
Ki67	-0.3922795
hormonal_therapy	17.4040569

```r
# Create table with model performance results
tibble(
  "RMSE" = rmse_A,
  "R^2" = r2_A
) |>
  kable()
```

RMSE	R^2
0.7470232	0.1674833

Part II

About a decade later, Researcher B conducted a separate study enrolling 2000 breast cancer patients diagnosed between 2000 and 2005, using the same study design and variable definitions. Like Researcher A's study, all patients were followed until disease progression or death. Researcher B's data are stored in datB.RData. Researcher A would like to evaluate whether the prediction model built on the original data generalizes well to Researcher B's dataset. Does the model generalize well? If yes, provide justification. If not, build a new prediction model for the 2000–2005 cohort and discuss what has changed in prediction.

Import dataset and create matrix

```
load("data/datB.RData")

# Remove id for matrix operations
all_datB <- datB |> select(-"id")
```

Evaluate the model's performance on researcher B's dataset

```
# Use all of datB to test generalizability of elastic net model
enet_pred_B <- predict(enet.fit, newdata = all_datB)

# Compare performances
rmse_B <- RMSE(enet_pred_B, all_datB$log_PFS)

# Report table comparing performance
performance_table <- data.frame(
  Dataset = c("datA (Internal Test Set)", "datB (External Validation)"),
  RMSE = c(round(rmse_A, 4), round(rmse_B, 4))
)

colnames(performance_table) <- c("Dataset", "RMSE")

performance_table |>
  kable(caption = "Elastic Net Model Performance: Internal vs External Validation") |>
  kable_styling(full_width = FALSE)
```

Table 5: Elastic Net Model Performance: Internal vs External Validation

Dataset	RMSE
datA (Internal Test Set)	0.7470
datB (External Validation)	0.8356

Creation of New Model

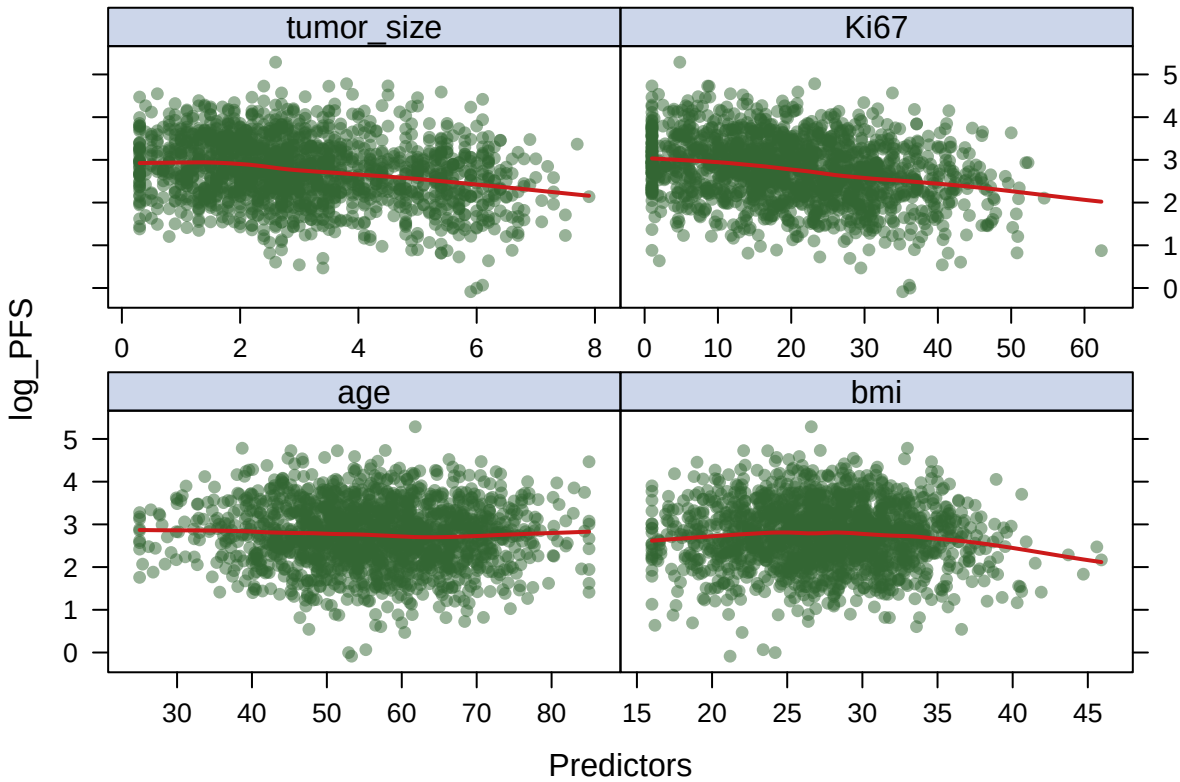
Import and partition data using 'tidymodels' package

```
set.seed(2)

datB_split = initial_split(datB, prop = 0.8)

# Extract the training and test data
training_datB = training(datB_split)
testing_datB = testing(datB_split)

# Training data
```

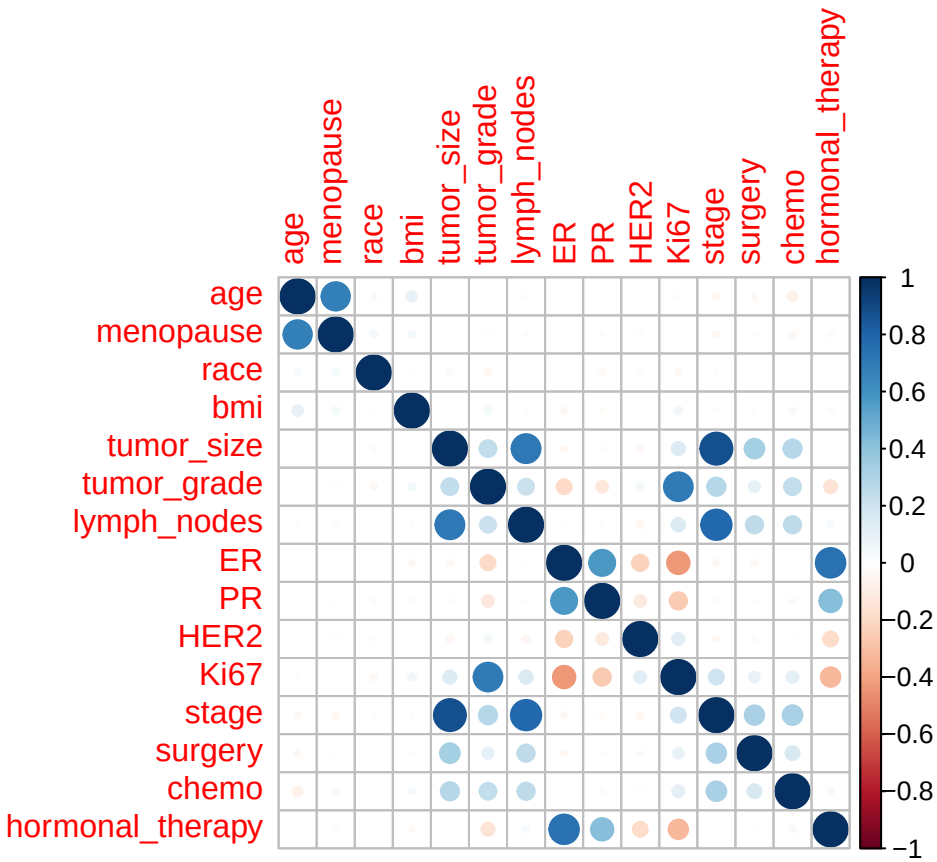
```r
x_trainB = model.matrix(log_PFS ~ . - id, training_datB)[, -1]
y_trainB = training_datB$log_PFS

# Test data
x_testB  = model.matrix(log_PFS ~ . - id, testing_datB)[, -1]
y_testB = testing_datB$log_PFS

# Remove ID
training_datB <- training_datB |>  select(-"id")
testing_datB  <- testing_datB  |> select(-"id")
```

Exploratory Data Analysis

```r
# Feature plots to look at trends between predictors and the outcome (log_PFS)
featurePlot(x = training_datB[ ,c(1, 4, 5, 11)],
            y = training_datB[ ,16],
            plot = "scatter",
            span = .5,
            labels = c("Predictors","log_PFS"),
            type = c("p", "smooth"),
            layout = c(2, 2))
```



```r
# Calculates the correlation matrix for all the predictors
corrplot(cor(x_trainB), method = "circle", type = "full")
```



Elastic Net model

```r
# Elastic Net

trainControl(method = "cv", number = 10)
```

```
## $method
## [1] "cv"
##
## $number
## [1] 10
##
## $repeats
## [1] NA
##
## $search
## [1] "grid"
##
## $p
## [1] 0.75
```

```
##
## $initialWindow
## NULL
##
## $horizon
## [1] 1
##
## $fixedWindow
## [1] TRUE
##
## $skip
## [1] 0
##
## $verboseIter
## [1] FALSE
##
## $returnData
## [1] TRUE
##
## $returnResamp
## [1] "final"
##
## $savePredictions
## [1] FALSE
##
## $classProbs
## [1] FALSE
##
## $summaryFunction
## function (data, lev = NULL, model = NULL)
## {
##     if (is.character(data$obs))
##         data$obs <- factor(data$obs, levels = lev)
##     postResample(data[, "pred"], data[, "obs"])
## }
## <bytecode: 0x000002498637ac40>
## <environment: namespace:caret>
##
## $selectionFunction
## [1] "best"
##
## $preProcOptions
## $preProcOptions$thresh
## [1] 0.95
##
## $preProcOptions$ICAcomp
## [1] 3
##
## $preProcOptions$k
## [1] 5
##
## $preProcOptions$freqCut
## [1] 19
##
```

```
## $preProcOptions$uniqueCut
## [1] 10
##
## $preProcOptions$cutoff
## [1] 0.9
##
##
## $sampling
## NULL
##
## $index
## NULL
##
## $indexOut
## NULL
##
## $indexFinal
## NULL
##
## $timingSamps
## [1] 0
##
## $predictionBounds
## [1] FALSE FALSE
##
## $seeds
## [1] NA
##
## $adaptive
## $adaptive$min
## [1] 5
##
## $adaptive$alpha
## [1] 0.05
##
## $adaptive$method
## [1] "gls"
##
## $adaptive$complete
## [1] TRUE
##
##
## $trim
## [1] FALSE
##
## $allowParallel
## [1] TRUE
```

```
ctrl1 <- trainControl(method = "cv", number = 10) # set up cv
set.seed(2) # set seed for reproduction

enet.test.b <- train(log_PFS ~ .,
                     data = training_datB,
                     method = "glmnet",
```

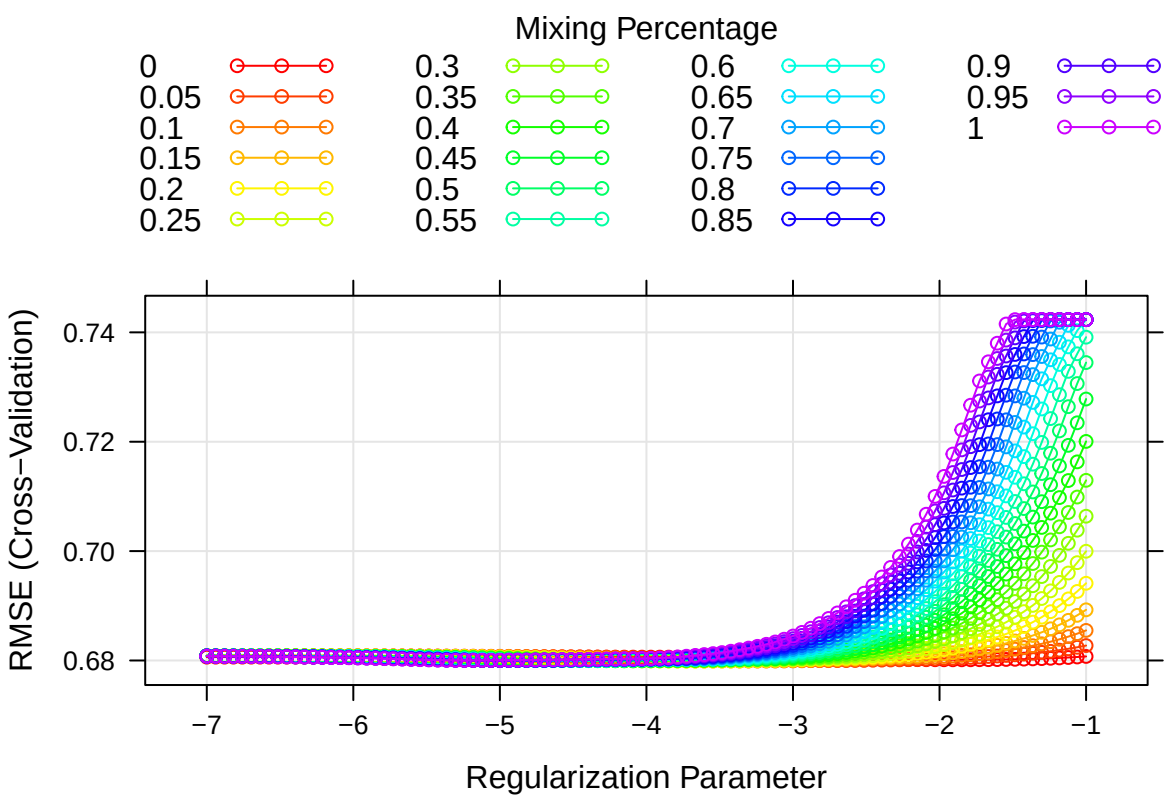
```
                    tuneGrid = expand.grid(alpha = seq(0, 1, length = 21),
                                           lambda = exp(seq(-1, -7, length = 100))),
                    trControl = ctrl1)
```

```
## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.
```

```
enet.test.b$bestTune
```

```
##     alpha    lambda
## 359  0.15 0.0306584
```

```
myCol <- rainbow(25)
myPar <- list(superpose.symbol = list(col = myCol),
superpose.line = list(col = myCol))

plot(enet.test.b, par.settings = myPar, xTrans = log)
```




```
# Coefficients in the final model
coef(enet.test.b$finalModel, enet.test.b$bestTune$lambda)
```

```
## 16 x 1 sparse Matrix of class "dgCMatrix"
##                    s=0.0306584
```

```
## (Intercept)       3.122084224
## age              -0.002852149
## menopause         .
## race              0.015769731
## bmi              -0.001204749
## tumor_size       -0.063034816
## tumor_grade      -0.050781111
## lymph_nodes      -0.018410630
## ER                0.159005621
## PR                0.094382985
## HER2              0.031095728
## Ki67             -0.005772904
## stage             .
## surgery          -0.017294705
## chemo            -0.031110872
## hormonal_therapy  0.190544270
```

```r
# Predict on testing_datB
enet.pred.b <- predict(enet.test.b, newdata = testing_datB)

rmse_testB <- RMSE(enet.pred.b, testing_datB$log_PFS)
r2_testB  <- R2(enet.pred.b, testing_datB$log_PFS)
```

```r
# Create table with the optimal tuning parameters for the chosen model
opt_para <- enet.test.b$bestTune
rownames(opt_para) <- NULL
opt_para |>
  kable()
```

alpha	lambda
0.15	0.0306584

```r
# Create a table with the model coefficients
coefs1 <- as.data.frame(as.matrix(coef(enet.test.b$finalModel, enet.test.b$bestTune$lambda)))
coefs1$Predictors <- rownames(coefs1)
colnames(coefs1)[1] <- "Coefficients"
coefs1 <- coefs1[, c("Predictors", "Coefficients")]
rownames(coefs1) <- NULL
coefs1 |>
  kable()
```

Predictors	Coefficients
(Intercept)	3.1220842
age	-0.0028521
menopause	0.0000000
race	0.0157697
bmi	-0.0012047
tumor_size	-0.0630348
tumor_grade	-0.0507811
lymph_nodes	-0.0184106

Predictors	Coefficients
ER	0.1590056
PR	0.0943830
HER2	0.0310957
Ki67	-0.0057729
stage	0.0000000
surgery	-0.0172947
chemo	-0.0311109
hormonal_therapy	0.1905443

```
# Create a table with the interpretable output
coefs2 <- as.data.frame(as.matrix(coef(enet.test.b$finalModel, enet.test.b$bestTune$lambda)))
coefs2$Predictors <- rownames(coefs2)
colnames(coefs2)[1] <- "Percent Change in PFS"
coefs2 <- coefs2[, c("Predictors", "Percent Change in PFS")]
coefs2 <- coefs2 |>
  filter(Predictors %in% c("(Intercept)", "age", "race", "bmi", "tumor_size", "tumor_grade", "lymph_node
                           "ER", "PR", "HER2", "Ki67", "surgery", "chemo", "hormonal_therapy"))
coefs2$`Percent Change in PFS`<- (exp(coefs2$`Percent Change in PFS`)-1)*100
rownames(coefs2) <- NULL
coefs2 |>
  kable()
```

Predictors	Percent Change in PFS
(Intercept)	2169.3628990
age	-0.2848085
race	1.5894729
bmi	-0.1204023
tumor_size	-6.1089216
tumor_grade	-4.9513301
lymph_nodes	-1.8242190
ER	17.2344536
PR	9.8980558
HER2	3.1584251
Ki67	-0.5756273
surgery	-1.7146010
chemo	-3.0631909
hormonal_therapy	20.9907935

```
# Create table with model performance results
tibble(
  "RMSE" = rmse_testB,
  "R^2" = r2_testB
) |>
  kable()
```

RMSE	R^2
0.6572825	0.1611754

Compare coefficients from datA & datB model

```r
# Extract coefficients from both models
coefs_A <- as.data.frame(as.matrix(coef(enet.fit$finalModel, enet.fit$bestTune$lambda)))
coefs_B <- as.data.frame(as.matrix(coef(enet.test.b$finalModel, enet.test.b$bestTune$lambda)))

# Rename columns
colnames(coefs_A) <- "Coef_datA"
colnames(coefs_B) <- "Coef_datB"

# Add predictor names
coefs_A$Predictor <- rownames(coefs_A)
coefs_B$Predictor <- rownames(coefs_B)

# Merge both into one table
coef_comparison <- merge(coefs_A, coefs_B, by = "Predictor", all = TRUE)

# Add a column showing the difference
coef_comparison$Difference <- coef_comparison$Coef_datB - coef_comparison$Coef_datA

# Round for readability
coef_comparison$Coef_datA <- round(coef_comparison$Coef_datA, 4)
coef_comparison$Coef_datB <- round(coef_comparison$Coef_datB, 4)
coef_comparison$Difference <- round(coef_comparison$Difference, 4)

# Reorder columns
coef_comparison <- coef_comparison[, c("Predictor", "Coef_datA", "Coef_datB", "Difference")]

# Sort by absolute difference to highlight what changed most
coef_comparison <- coef_comparison[order(abs(coef_comparison$Difference), decreasing = TRUE), ]

rownames(coef_comparison) <- NULL

# Display table
coef_comparison |>
  kable(caption = "Comparison of Elastic Net Coefficients from datA and datB Models",
        col.names = c("Predictor", "datA Coefficient", "datB Coefficient", "Difference")) |>
  add_footnote("Coefficients are on the log-PFS scale, Difference = datB minus datA.
               Variables with zero coefficients were excluded by elastic net regularization.",
               notation = "none")
```

Table 10: Comparison of Elastic Net Coefficients from datA and datB Models

Predictor	datA Coefficient	datB Coefficient	Difference
(Intercept)	2.7755	3.1221	0.3466
HER2	-0.1773	0.0311	0.2084
PR	0.0000	0.0944	0.0944
lymph_nodes	-0.0545	-0.0184	0.0361
chemo	0.0000	-0.0311	-0.0311
hormonal_therapy	0.1605	0.1905	0.0301
tumor_grade	-0.0801	-0.0508	0.0293

Predictor	datA Coefficient	datB Coefficient	Difference
ER	0.1815	0.1590	-0.0225
race	0.0000	0.0158	0.0158
surgery	-0.0051	-0.0173	-0.0122
tumor_size	-0.0536	-0.0630	-0.0094
Ki67	-0.0039	-0.0058	-0.0018
bmi	0.0000	-0.0012	-0.0012
age	-0.0026	-0.0029	-0.0002
menopause	0.0000	0.0000	0.0000
stage	0.0000	0.0000	0.0000

Note: Coefficients are on the log-PFS scale, Difference = datB minus datA. Variables with zero coefficients were excluded by elastic net regularization.